

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 07

বক্তৃতা – 33

থ্রেড – লাইট ওয়েট প্রসেস (Thread – Light Weight Process) Part 1

এই বিশেষ সমস্যাটি বিবেচনা করুন। যদি একজন 10 দিনে একটি কাজ করে তাহলে 10 জনকে একই কাজ করতে কতদিন লাগবে? সুতরাং, আমরা আমাদের স্কুলের দিনগুলিতে এই ধরনের সমস্যাগুলি সম্পন্ন করেছি, এবং উত্তর গণনা করা বেশ সহজ। মূলত এটি কাজ করতে 10 জনের 1 দিন নিতে হবে। এখন কাজ আরও বেশি করে করা হয় কারণ 10 জন সমান্তরালভাবে কাজ করছেন। সুতরাং, প্রতিটি মানুষের পুরো কাজ এক দশমাংশ প্রয়োজন এবং পুরো কাজ শুধু একটি একক দিনের মধ্যে সম্পন্ন হয়। সুতরাং, parallelization এর এই একই ধারণাটি কম্পিউটার সিস্টেমে ব্যাপকভাবে প্রয়োগ করা হয়। মূলত, application গুলির কার্যকারিতা উন্নত করার জন্য কম্পিউটার সিস্টেমে সমান্তরালভাবে বেশিরভাগ সময় ব্যবহার করা হয়। সুতরাং, কি করা হয় যে যদি একটি বড় কাজ করতে হয়, আমরা সিস্টেমের মধ্যে এটি সমান্তরাল হবে। তাই বিভিন্ন computing entity , এই computing entity গুলি খুবই ছোট এবং বৃহত্তর কাজের একটি ছোট অংশ করে এবং এই computing entity গুলি সমান্তরালে চলে এবং সমস্ত একসাথে কাজটি আরও দ্রুত সম্পন্ন হবে। সুতরাং, আজকাল parallelization ব্যাপকভাবে বেশ কয়েকটি কম্পিউটার হার্ডওয়্যার দ্বারা সমর্থিত। প্রকৃতপক্ষে, কিছু হার্ডওয়্যার রয়েছে যাকে বলা হয় GPUs বা Graphics Processing Units যা সমানভাবে হাজার হাজার কাজ করতে সক্ষম। সুতরাং, এই প্রতিটি কাজ একটি বড় কাজের একটি খুব ছোট অংশ, কিন্তু এই সমস্ত কাজ সমান্তরালে করা হয় যে বড় বড় কাজ সম্পূর্ণ করার জন্য আরো কম সময় অর্জন করবে সমান্তরাল করার সময়। এখন

parallelization এর একটি গুরুত্বপূর্ণ construct হল threading বা thread, thread মূলত execution entity যা পূর্বের ভিডিওগুলির মধ্যে আমরা আলোচনা করেছি। যাইহোক, thread অত্যন্ত light weight process, এবং তারা মূলত parallelization আরো অনেক সহজ করার জন্য ব্যবহৃত হয়। এই ভিডিওতে আমরা thread গুলি দেখতে পাব এবং মূলত thread গুলির ভূমিকা দেব। তাই ভিডিওটি কিভাবে thread তৈরি করা এবং ধ্বংস করা হবে, কীভাবে প্রসেস থেকে এটি আলাদা এবং কিভাবে বিভিন্ন অপারেটিং সিস্টেম বিভিন্ন উপায়ে thread সমর্থন করে তা বর্ণনা করবে।

(স্লাইড সময় : 03:12 দেখুন)

তাই আসুন একটি উদাহরণ সঙ্গে এই নির্দিষ্ট ভিডিও শুরু করা যাক। সুতরাং আসুন আমরা এই বিশেষ প্রোগ্রামটি বিবেচনা করি, মূলত এখানে একটি ফাংশন আছে addall যা main থেকে call করা হয় এবং unsigned long return করে। সুতরাং, এই ফাংশন addall করে, শুধু প্রথম 10 million positive integer যোগ করে এবং এখানে যোগফল return করা হয়, এবং main ফাংশনে print হয়। তাই স্পষ্টতই, এই বিশেষ প্রোগ্রামটি অনেক বেশি ভিন্ন এবং আরো অনেক সহজ পদ্ধতিতে লেখা যেতে পারে, 0 থেকে 10 million প্রতিটি সংখ্যা iterate না করে। যাইহোক, thread বোঝার জন্য, আসুন আমরা এই বিশেষ প্রোগ্রামের সাথে যাই। তাই, আগের ভিডিওগুলিতে আমরা যা দেখেছি তা হল যে আমরা যখন এই প্রোগ্রামটি লিখব, তখন আমরা একটি compiler ব্যবহার করি যেমন gcc যা executable তৈরি করে এবং তারপর executable টি একটি প্রক্রিয়ায় তৈরি করা যায়। সুতরাং, আমরা শেষ পর্যন্ত একটি প্রক্রিয়া যা তার নিজস্ব memory space থাকবে এবং এর অংশটি RAM-র মধ্যে হতে পারে এবং এখন আমাদের দেখতে হবে যদি এই প্রক্রিয়াটি 4 টি ভিন্ন প্রসেসরের সাথে একটি সিস্টেমে কার্যকর করা হয় তবে কি হবে? সুতরাং, কি হয় যে অপারেটিং সিস্টেমের scheduler এই প্রক্রিয়াটি বাছতে এবং এই প্রসেসরগুলির মধ্যে একটিতে assign করবে। সুতরাং, সবসময় একবার এই প্রক্রিয়া এই প্রসেসরের মধ্যে একটিতে execute হবে, এবং খুব কমই এই প্রক্রিয়া অন্যান্য প্রসেসরে যেতে পারে, কিন্তু আমরা এ নিয়ে ভাবছি না। এখন, আমরা এখানে যা দেখি তা হল যে আমাদের সিস্টেমে 4 টি প্রসেসর উপস্থিত রয়েছে যদিও আমরা এই সিস্টেমটি চালানোর জন্য যথেষ্ট সময় ব্যয় করি। শুধু একটি প্রসেসর চলে এবং অন্যান্য 3 টি প্রসেসর কর্মহীন বা অন্য কিছু কাজ করে। সুতরাং এই সমগ্র প্রোগ্রামের execution এর সময়টি বেশ বড় হবে কারণ এই যোগফল ক্রমানুসারে 1 থেকে 10 million পর্যন্ত পাওয়া যায় যেমন এই লুপে হয়। সুতরাং, আমরা 4 টি প্রসেসরের সাথে এই হার্ডওয়্যার আছে, এই একটি খুব ভাল উপায় না বা এই বিশেষ প্রোগ্রাম লিখতে কার্যকর উপায় নয়। সুতরাং, আমরা এই প্রোগ্রাম parallelizable করে ভাল করতে পারি এবং এই 4 টি প্রসেসরে চালানো যেতে পারে।

(স্লাইড সময় : 06:07 দেখুন)

সুতরাং, আমরা কি করবো তা হল আমরা 10 মিলিয়ন সংখ্যা নিয়ে 4 দিয়ে ভাগ করে 2.5 মিলিয়নের 4 টি ভাগ করব। তাই, আমরা 4 টি প্রসেস তৈরি করব এবং প্রতিটি প্রক্রিয়াকে এই সংখ্যাগুলির এক চতুর্থাংশের মধ্য থেকে লুপ দেব; এর অর্থ হল, প্রথম প্রক্রিয়াটি 0 থেকে 2.5 মিলিয়ন এর মধ্যে লুপের মাধ্যমে এবং এই সমস্ত সংখ্যার সমষ্টি বের করবে, প্রক্রিয়া 2 2.5 মিলিয়ন থেকে 5 মিলিয়নের সংখ্যা যোগ করবে, প্রক্রিয়া 3 সংখ্যাগুলি যোগ করবে 5 মিলিয়ন থেকে 7.5 মিলিয়ন এবং প্রক্রিয়া 4 সংখ্যা যোগ করবে 7.5 মিলিয়ন থেকে 10 মিলিয়ন পর্যন্ত। তাই আমরা যদি এইভাবে 4 টি প্রসেস তৈরি করতে সক্ষম হই, তাহলে যখন আমরা এই 4 টি প্রসেস চালাই, এটি এইরকম কিছু হবে। সুতরাং, এই 4 টি প্রসেসের প্রতিটি সিস্টেমের মধ্যে একটি ভিন্ন প্রসেসরে চালানো হয় এবং এই প্রতিটি প্রক্রিয়াগুলি মোট মিলিয়ন সংখ্যার সমগ্র পরিমাণ যোগফলের পরিবর্তে 2.5 মিলিয়ন সংখ্যা যোগ করার একটি ছোট কাজ করে। উপরন্তু, প্রতিটি প্রক্রিয়া তার নিজস্ব প্রসেসরে নির্ধারিত হয়, তাই, আমরা parallelization পাই। সুতরাং আমরা কি আশা করব যে 10 মিলিয়ন সংখ্যার সমষ্টি খুঁজে পেতে সমগ্র প্রোগ্রাম সম্পন্ন করা যাবে 4 গুণ দ্রুত। সুতরাং, এটি আমাদের parallelization সমস্যা সমাধান করবে এবং এটি আরো কার্যকর পদ্ধতিতে উপস্থিত প্রসেসরের ব্যবহার করা হবে। এবং আমাদের এই বিশেষ পদ্ধতির কিছু বৈশিষ্ট্য নিম্নরূপ হয়। সুতরাং, এই বিশেষ 4 টি প্রসেস তৈরি করার জন্য আমাদের 4 টি fork system call প্রয়োজন হবে। সুতরাং, প্রতিটি fork system call এই প্রক্রিয়াগুলির মধ্যে একটি তৈরি করবে। উপরন্তু আমরা parallelization বা speed up এর এই বিশেষ মডেলে অন্য ধর্ম লক্ষ্য করি যে, এই প্রক্রিয়াগুলি প্রতিটি একে অপরের থেকে বিচ্ছিন্ন হয়। প্রতিটি প্রক্রিয়ার নিজস্ব instruction, data, heap, stack আছে এবং এই প্রক্রিয়ার মধ্যে এই segment গুলি এই অন্যান্য প্রক্রিয়ার থেকে বিচ্ছিন্ন হয়। পরবর্তী কারণে এই বিচ্ছিন্নতার জন্য inter process communication কৌশল প্রয়োজন, যা আমরা দেখেছি পূর্ববর্তী ভিডিওতে একটি প্রক্রিয়ার সাথে অপর প্রক্রিয়ার যোগাযোগ করার জন্য। তাই, আমরা দেখেছি IPC গুলি বিভিন্ন উপায়ে অর্জন করা যায়। সুতরাং, এক উপায় হল send এবং receive system call এর মাধ্যমে অপারেটিং সিস্টেম দ্বারা একটি চ্যানেল তৈরি করতে সাহায্য করবে যার মাধ্যমে IPC অর্জন করা সম্ভব। অন্যান্য উপায়গুলি shared memory দ্বারা হয়, যা এই প্রক্রিয়ার মধ্যে একটি shared memory তৈরি করে আবার system call ব্যবহার করে এবং এই shared memory তখন প্রসেসগুলির মধ্যে IPCs এর জন্য ব্যবহার করা যেতে পারে। উপরন্তু, প্রক্রিয়া বিভিন্ন পর্যায়ে আছে, যেমন creation, deletion বা file operation ইত্যাদি। সুতরাং, এই সমস্ত প্রক্রিয়া ব্যবস্থাপনা কার্যক্রমগুলি system call এর মাধ্যমে সম্পন্ন করা হয়। এবং উল্লিখিত শেষ বিষয়টি, প্রত্যেক প্রক্রিয়ার তার নিজস্ব বিচ্ছিন্ন memory map আছে যাতে instructions, data, heap এবং stack থাকে। এখন, এই বৈশিষ্ট্যগুলির মধ্যে আমরা যা দেখেছি তা হল এই সমস্যাটি সমাধান করার সাথে জড়িত অনেক system call আছে যা parallelization সম্পাদনের জন্য একাধিক প্রসেস করে। এবং আমরা আগের ভিডিওতে দেখেছি যে এই system call গুলির প্রচুর পরিমাণে

overhead রয়েছে। তাই, প্রতিটি সময় এই প্রক্রিয়াগুলি একটি system call করে সেটি অপারেটিং সিস্টেমে execute হয়, একটি software trap এর কারণে এবং অপারেটিং সিস্টেম দেখে কোন system call invoke হয়েছে এবং সেই system call handler execute হবে। সুতরাং, আমরা একটি পূর্ববর্তী ভিডিওতে দেখা হিসাবে এই পুরো প্রক্রিয়া বেশ উল্লেখযোগ্য হতে পারে। অন্য একটি overhead এই কারণে হয় যে 4 টি প্রসেসের একটি বড় অংশ অনুরূপ। উদাহরণস্বরূপ, এই সমস্ত 4 টি প্রসেসগুলির একই instructions যা তারা পরিচালনা করে এবং একই array হতে পারে যা তারা access করছে, একই global data যা তারা access করছে এবং সেইজন্য, আমরা যা দেখি তা হল যে অনেকগুলি সদৃশ instruction এবং data একাধিক প্রসেস যা সমান্তরাল একটি কাজ থাকার কারণে থাকছে। সুতরাং, আমাদের এর চেয়ে ভাল করতে করার উপায় আছে? কি সমাধান আছে, যেখানে parallelization উপস্থিত overheads পরিমাণ হ্রাস করা যাবে? সুতরাং, এটা অর্জন করার জন্য, ব্যবহৃত হয় threading এর ধারণা।

(স্লাইড সময় : 12:02 দেখুন)

সুতরাং, আগের মত threading এর জন্য, আমরা 10 মিলিয়ন কে 4 টি ভাগ করি যার প্রতিটি 2.5 মিলিয়ন। কিন্তু 4 টি প্রসেস তৈরি করার পরিবর্তে আমরা আগে যেমন করেছি, আমরা কেবল একটি প্রক্রিয়া তৈরি করব এবং এই প্রক্রিয়ার মধ্যে আমরা একটি thread বলে execution entity তৈরি করি। সুতরাং, এই thread গুলির প্রত্যেকটি আগের মতই সমান্তরাল প্রসেসগুলির ব্যবহার করে, একটি লুপ থাকবে যা এই thread গুলির প্রত্যেকটির এক-চতুর্থাংশের মধ্যে 2.5 মিলিয়ন সংখ্যার যোগফল পাবে। সুতরাং, চিত্রে এই চেহারা হবে। সুতরাং, আমাদের 1 টি প্রক্রিয়া আছে, সুতরাং, এই 1 টি প্রক্রিয়া বলতে হবে যে এর একটি instruction set, একটি global data, heap এবং অন্যান্য segment রয়েছে এবং এই প্রক্রিয়ার মধ্যে 4 টি execution content আছে। এবং এই thread এর প্রতিটি একটি ভিন্ন প্রসেসরের মধ্যে স্থাপনিত হয়। উদাহরণস্বরূপ, লাল thread টি প্রসেসর 1 এ যায় এবং এখানে এটি execute করা হয়, নীল thread টি প্রসেসর 2 তে যায়, সবুজ এবং হলুদ প্রসেসর 3 এবং 4 এ যথাক্রমে যায়। সুতরাং আমরা thread ব্যবহার করে parallelization অর্জন করতে সক্ষম। সুতরাং, এই সমান্তরালতা পূর্ববর্তী স্লাইডে আমরা যা দেখেছি তা সমান্তরাল প্রক্রিয়ার সমতুল্য। যাইহোক, এটাতে overhead কম আছে। সুতরাং, একটি parallelization এই বিশেষ thread মডেলের বৈশিষ্ট্য, নিম্নরূপ হয়। যেহেতু আমরা এখানে একটিমাত্র প্রক্রিয়া তৈরি করেছি, তাই আমাদের যা দরকার তা হল এই প্রক্রিয়াটি তৈরি করার জন্য শুধুমাত্র একটি fork system call লাগে। এখন 4 টি thread তৈরি করার জন্য এই নির্দিষ্ট প্রক্রিয়ার মধ্যে, আমরা একটি library call ব্যবহার করতে পারি যেমন Pthread create যার system call এর জন্য প্রয়োজন হতে পারে বা নাও হতে পারে। উপরন্তু, আগের স্লাইড থেকে প্রক্রিয়া parallelization সঙ্গে একটি বড় পার্থক্য হল যে এই thread একে অপরের থেকে বিচ্ছিন্ন হয় না। তাই, এর মানে হল এই যে এই সমস্ত 4 টি থ্রেড একই প্রক্রিয়ায় execute হয়, তাই, এই

thread প্রসেসের segment যেমন heap segment এবং global data access করতে পারে। এছাড়াও এটি সম্ভব যে এই প্রক্রিয়ায় instruction বা text segment বিভিন্ন thread গুলির মধ্যে ভাগ করা হয়। এখন এই প্রতিটি thread এর নিজস্ব stack থাকবে এবং এর নিজস্ব stack এর নিজস্ব execution context থাকবে। সুতরাং, প্রতিটি thread local variable তৈরি করতে এবং সেইসাথে function call গুলির সম্পর্কে তথ্য সংরক্ষণ করতে stack ব্যবহার করতে সক্ষম হবে। উদাহরণস্বরূপ, এই thread গুলির execution এর জন্য, যেমন অন্য thread এর জন্য অপেক্ষা করা বা একটি thread প্রস্থান করার জন্য, system call করতে নাও হতে পারে। তাই, এখন আমরা এই thread গুলি আরও বিস্তারিতভাবে দেখব।

(স্লাইড সময় : 15:53 দেখুন)

সুতরাং আসুন আমরা এই বিশেষ চিত্রটি দেখি। সুতরাং, আমরা এখানে এই একক প্রক্রিয়া আছে এবং এই বিশেষ প্রক্রিয়ার মধ্যে একাধিক thread আছে। সুতরাং, এই প্রক্রিয়াটির execution context এটি। সুতরাং, এই execution context এ একই কোড share করে, এবং একই global data access এবং heap আছে, এবং এছাড়াও একই file access থাকবে। সুতরাং, অন্যটি থেকে একটি thread কি করে পৃথক হবে তা হল প্রতিটি thread এর নিজস্ব stack আছে যা local variable এর জন্য এবং function call এর জন্য ব্যবহার করে। সুতরাং এই thread গুলির execution context যেমন উল্লেখ করা হয়েছে এবং এই thread গুলির প্রতিটি সিস্টেমে বিদ্যমান প্রসেসরের একটিতে চালানোর জন্য scheduled হতে পারে। অথবা এটি সম্ভব যে এই thread বিভিন্ন সময়ে একই প্রসেসরের উপর scheduled হতে পারে। এখন প্রতিটি thread একে অপরের প্রায় স্বাধীনভাবে কাজ করছে, তাই এখানে এই register একটি thread এর execution context নির্দেশ করে। তাই, আমরা দেখেছি যে, প্রতিটি thread এর নিজস্ব execution context রয়েছে। তাই এই thread এর execution context এ কয়েকটি register আছে যেমন EIP এবং ESP register। অনুরূপভাবে এই thread এর execution context এ stack এবং register values রয়েছে। সুতরাং, এই register values একটি context switch এর পরে বা সময় thread থামাতে এবং পুনরায় আরম্ভ করতে সাহায্য করবে। এখন প্রসেসের চেয়ে thread থাকার প্রধান সুবিধা কি?

(স্লাইড সময় : 17:52 দেখুন)

সুতরাং এটি আসলে এই বিশেষ স্লাইড থেকে দেখা যায়; মূলত সবচেয়ে গুরুত্বপূর্ণ পার্থক্য প্রসেসগুলির তুলনায় thread অত্যন্ত light weight হয়। একটি উদাহরণ নিতে আমাদের এই বিশেষ টেবিলের উপর নজর রাখুন, যা fork system call ব্যবহার করে 50,000 প্রসেস তৈরি করতে সময় দেখায় এবং এটি একই মেশিনে 50,000 thread তৈরির সাথে তুলনা করে। সুতরাং, আসুন আমরা এই মেশিনগুলির কোন একটি উদাহরণ হিসাবে নিয়ে যাই। আসুন আমরা দেখি এবং যদি এই বর্ণনায় দেখি তবে এই নির্দিষ্ট column দেখি, যা দেখায় যে fork system call করতে 8.1 সময় লাগে 50,000 প্রসেস তৈরি করতে। অন্যদিকে, ঠিক একই মেশিনে 50,000 thread তৈরি

করতে শুধুমাত্র 0.9 সময় লাগবে। সুতরাং আমরা দেখি যে thread তৈরি করতে, প্রক্রিয়াগুলি তৈরির তুলনায় অনেক বেশি দক্ষ। অনুরূপভাবে যদি আমরা অন্য অপারেশনগুলিকেও বিশ্লেষণ করি তবে আমরা বুঝতে পারি যে thread সৃষ্টি ও পরিচালনা প্রসেস পরিচালনার তুলনায় অনেক বেশি হালকা হয়। Thread গুলির আরেকটি গুরুত্বপূর্ণ সুবিধা হল যে তারা execution entity এর মধ্যে দক্ষ যোগাযোগ সম্ভব করে। উদাহরণস্বরূপ, আমরা প্রক্রিয়ার মধ্যে দেখেছি যেহেতু প্রত্যেকটি প্রক্রিয়া একে অপরের থেকে বিচ্ছিন্ন হয়ে যায়, তাই প্রসেসগুলি যোগাযোগ করতে পারে এমন একমাত্র উপায় হল IPC দ্বারা, এবং যেমন আমরা দেখেছি IPC এর জন্য বেশ কিছুটা system call প্রয়োজন। সুতরাং, উদাহরণস্বরূপ, IPC message passing এ send এবং receive একটি system call করে এবং এর যথেষ্ট overhead আছে। একইভাবে shared memory IPC গুলির জন্য, shared memory তৈরি ও পরিচালনা করতে system call লাগে যার আবার overhead রয়েছে। অন্যদিকে, একটি প্রসেসের মধ্যে দুটি thread এর মধ্যে যোগাযোগ খুব সহজেই করা যায়, এর কারণ হচ্ছে প্রক্রিয়াটির global data বিভিন্ন thread এর মধ্যে shared করা হয় এবং এর মানে হল যে প্রতিটি thread প্রসেসের global data access করতে পারে। অতএব, একটি প্রক্রিয়ার মধ্যে thread গুলির মধ্যে যোগাযোগ shared global data দ্বারা বা একইভাবে প্রক্রিয়ার shared heap মাধ্যমে সহজে অর্জিত হতে পারে। Thread এর আরেকটি বড় সুবিধা হল যে এটিতে কার্যকর context switching হয়। সুতরাং, যখন আমরা একটি প্রক্রিয়া থেকে অন্যটিতে সুইচ করি, context switch এর সময় উল্লেখযোগ্য হয় কারণ প্রক্রিয়াটির state টি বেশ বড়। এছাড়াও আমাদের TLB প্রয়োজন, আমরা flush করি এবং নতুন প্রক্রিয়ার page table সক্রিয় হয় এবং এই প্রক্রিয়াটি আর এই নতুন page table তারপর TLB এর মধ্যে লোড হবে, ইত্যাদি। অন্যদিকে, একটি thread থেকে অন্যটায় সঞ্চার করার জন্য ক্ষুদ্রতর context রক্ষা করা প্রয়োজন, কারণ thread এর context একটি প্রক্রিয়ার চেয়ে অনেক ছোট। অতএব, থ্রেডগুলির মধ্যে switch করার সময় কম সংখ্যার সংরক্ষণ করা প্রয়োজন। অতএব, thread এর context switch দ্রুত হবে। এবং অন্য যে কারণে thread এর context switch দ্রুত হয় যে page table পরিবর্তন করা হয় না। মূলত thread গুলি একই প্রক্রিয়া থেকে চালানো হয় এবং প্রতিটি প্রক্রিয়াকে সরাসরি page এর একটি সেট এবং page table এর সঙ্গে যুক্ত করা হয়। একটি thread থেকে অন্য thread switch করার সময়, এই page table এবং page directory একই থাকবে। অতএব, কোনও অন্য page table সক্রিয় করার কোনও overhead নেই এবং সেইজন্য, locality একই থাকে TLB এর cache entry বৈধ থাকে।

(স্লাইড সময় : 22:27 দেখুন)

তাই এই বিশেষ স্লাইড প্রসেস এবং thread গুলির মধ্যে অনেক পার্থক্য পুনরাবৃত্তি করে, এবং আমরা দ্রুত এই মাধ্যমে যাব। সুতরাং, মূলত একটি প্রক্রিয়া হল একটি বিচ্ছিন্ন executed entity যার নিজস্ব code, heap, stack এবং অন্যান্য অংশ রয়েছে। অন্য দিকে, একটি thread এর নিজস্ব data segment এবং heap নেই এবং মূলত এটি

প্রসেসের data segment এবং heap ব্যবহার করে। সুতরাং, একটি নির্দিষ্ট প্রক্রিয়ার মধ্যে সমস্ত thread একই data segment এবং heap ব্যবহার করবে। আরেকটি পার্থক্য হল একটি প্রক্রিয়া একটি সম্পূর্ণ entity। সুতরাং একটি প্রক্রিয়াকে আপনি একটি একক thread হিসাবে বিবেচনা করতে পারেন যে একটি একক execution context এর উপর বেঁচে থাকতে পারে অন্য দিকে; একটি thread থ্রেড তার নিজের উপর বাস করতে পারে না। সুতরাং, এটি একটি প্রসেসের সঙ্গে সংযুক্ত করা প্রয়োজন। সুতরাং, যখন একটি প্রক্রিয়া শেষ হয় তখন সেই সব thread গুলি যে প্রক্রিয়াতে উপস্থিত হয় তাও শেষ হবে। সুতরাং, একটি প্রক্রিয়া ছাড়া অন্য কথায়, একটি thread চালানো যাবে না। অন্য দিকে, একটি প্রক্রিয়ায় একটি একক thread থাকবে এবং এটি প্রক্রিয়াটি চালানোর জন্য যথেষ্ট। সুতরাং, কেন আমরা একটি একক thread বলি? এর অর্থ কোনও একটি প্রক্রিয়ার execution context একক thread গঠন করে। সুতরাং, একটি প্রসেসের মধ্যে thread একই code এবং file share করে যেমন আমরা আগের স্লাইডে দেখেছি, কিন্তু প্রতিটি thread এর নিজস্ব stack থাকে এবং thread টি শেষ হলে এই stack টি পুনরুদ্ধার হয়। সুতরাং, এটি সম্ভব একটি থ্রেড সমাপ্ত হলে প্রক্রিয়াটি বাকি থাকে তা চলতে থাকবে। উদাহরণস্বরূপ, যদি একটি প্রক্রিয়া 10 টি thread দেয় এবং thread গুলির মধ্যে একটি সমাপ্ত হয় তবে বাকি 9 টি thread চালানো অব্যাহত থাকবে। অন্যদিকে যদি কোন প্রক্রিয়া বন্ধ হয়ে যায়, তবে সেই প্রক্রিয়ার মধ্যে সমস্ত thread বন্ধ হয়ে যাবে।

(স্লাইড সময় : 24:50 দেখুন)

সুতরাং, কিভাবে আমরা thread তৈরি এবং পরিচালনা করি? এখন এমন কিছু লাইব্রেরি আছে যা আমরা করতে পারি, কিন্তু আমরা এই বিশেষ ভিডিওটি দেখব কীভাবে pthread লাইব্রেরি নামে পরিচিত একটি খুব জনপ্রিয় লাইব্রেরী। সুতরাং, আমরা pthread লাইব্রেরির খুব গুরুত্বপূর্ণ ফাংশন কিছু দেখব যা thread তৈরি এবং পরিচালনা করতে ব্যবহৃত হয়। সুতরাং, আসুন দেখি কিভাবে একটি প্রক্রিয়াতে আমরা একটি thread তৈরি করতে পারি। সুতরাং, pthread লাইব্রেরিটিতে একটি thread তৈরি করা হয় এই ফাংশন pthread create মাধ্যমে যা বিভিন্ন argument গুলি ব্যবহার করে। সুতরাং, এটির 4 টি argument লাগে এবং এই 4 টি argument এখানে দেখানো হয়। সুতরাং প্রথম argument যা pthread underscore t টিতে pointer হয় তা হল thread identifier বা TID। সুতরাং, এটি process identifier বা PID এর অনুরূপ যা আমরা প্রক্রিয়া ভিডিওতে অধ্যয়ন করেছি। pthread create এর দ্বিতীয় argument হল attribute। তাই এই attribute এর মাধ্যমে তৈরি করা হয়েছে যে thread এর বিভিন্ন attribute নির্দিষ্ট করতে পারে। সুতরাং, pthread underscore create ফাংশন করা হবে যে এটি invoke হয় যখন, এটি একটি thread context বা নতুন thread context তৈরি করে যাচ্ছে এবং এটি এখানে নির্দিষ্ট এই ফাংশন থেকে যে নতুন context execute করা শুরু করে। তাই এই তৃতীয় parameter টি start underscore routine হিসাবে নির্দিষ্ট একটি ফাংশন একটি pointer যা নতুন thread context চালানো শুরু

করে। তাই এই ফাংশন একটি ভিন্ন thread মধ্যে চালানো শুরু হবে। pthread create এর সর্বশেষ argument হল arg বা a r g যা এই বিশেষ start routine ফাংশনের argument গুলির একটি pointer। তাই, পরবর্তী দুটি স্লাইডে, আমরা pthread create কিভাবে ব্যবহার করে তার উদাহরণ এবং pthread এর অন্যান্য ফাংশনের উদাহরণ দেখব। তাই এখন, একটি pthread তৈরি করেছি এবং আমাদের এটি তার কাজ করছে বলা যাক। পরের জিনিস আসলে একটি thread ধ্বংস করা এবং এই pthread underscore exit এই ফাংশন দ্বারা সম্পন্ন হয়। সুতরাং thread এর return status পাস করার জন্য, pthread underscore exit একটি return value এর pointer পাঠাবে। সুতরাং, এটি প্রক্রিয়ার মধ্যে exit system call এর মতো অনেকগুলি।

(স্লাইড সময় : 27:28 দেখুন)

এখন, আরেকটি গুরুত্বপূর্ণ pthread লাইব্রেরির ফাংশন হল এই বিশেষ ফাংশন কল pthread underscore join । তাই এই মাধ্যমে, একটি প্রক্রিয়া বা একটি thread একটি নির্দিষ্ট thread সম্পূর্ণ হতে অপেক্ষা করতে পারে। সুতরাং, এটি প্রক্রিয়াগুলির সাথে সম্পর্কিত wait system call এর মতই। সুতরাং, pthread underscore join নির্দিষ্ট করা হয় thread এর thread ID এবং এটি প্রাপ্ত করা হয় wait thread এর জন্য TID নির্দিষ্ট হয় thread exit status এ পাওয়া যায়। এই pthread underscore join calling thread কে অবরোধ করবে, যতক্ষণ না এই TID দ্বারা নির্দিষ্ট thread টি প্রস্থান করে এবং যখন thread টি প্রস্থান করে তখন pthread underscore join wake up হবে এবং এটি TID দ্বারা নির্ধারিত thread এর exit status পড়তে সক্ষম হবে। সুতরাং, এই নির্দিষ্ট তিনটি ফাংশন pthread underscore create, pthread underscore exit এবং pthread underscore join; আসুন দেখি কিভাবে আমরা এই pthread গুলি ব্যবহার করতে পারি যাতে আমাদের 10 মিলিয়ন ধনাত্মক পূর্ণসংখ্যার সমষ্টি পেতে parallelize করা যায়।

(স্লাইড সময় : 28:48 দেখুন)

সুতরাং, এটি করতে এই কোড হয়। সুতরাং, এই বিশেষ কোডটি কি এই যে এটি 4 টি thread তৈরি করে এবং প্রতিটি থ্রেড একটি 2.5 মিলিয়ন সংখ্যার যোগফল করে। এবং এই 4 টি thread সমান্তরালে চালানো হয় এবং সেইজন্য, খুব দ্রুত 2.5 মিলিয়ন নম্বর পাওয়া যায় এবং তারপর আমরা প্রথম 10 মিলিয়ন সংখ্যার সমষ্টি পেতে ফলাফল যোগ করি। তাই আসুন আরো বিস্তারিতভাবে এই উদাহরণটি দেখুন। সুতরাং, প্রথমত আমরা 4 টি pthread underscore t প্রকারের variable সংজ্ঞায়িত করেছি। সুতরাং, এইগুলি T1, T2, T3 এবং T4 হয়। তাই, এইগুলি 4 টি thread তৈরি করতে ব্যবহার করা হচ্ছে এবং এই TID গুলি thread ID ধারণ করবে। এখন আমরা আগের স্লাইডে দেখেছি, thread তৈরি কর হয় এই ফাংশন কল দ্বারা pthread underscore create। সুতরাং, এই প্রতিটি pthread underscore create invocations এই এক যে T1, T2, T3 এবং T4 যা thread identifier বা নতুন তৈরি thread এর TID পূরণের সঙ্গে pointer pass পাস করবে। দ্বিতীয় parameter টি হল thread টির



বৈশিষ্ট্যগুলি null হিসাবে উল্লেখ করা হয় যখন তৃতীয় parameter টি ফাংশন পয়েন্টার, যা যে ফাংশনে যেখানে thread টি চালানো উচিত তা নির্দেশ করে। সুতরাং, এখানে ফাংশন pointer এই ফাংশন thread pointer thread underscore fn, যা এখানে সংজ্ঞায়িত করা হয় এবং এখানে একটি argument void star নেয় যা এখানে নির্দিষ্ট করা হয়। সুতরাং, pthread underscore create করা হয়, যার শেষ argument এই বিশেষ ফাংশন এর arg এখানে পাস করা হবে। সুতরাং, এই পদ্ধতিতে যখন pthread underscore create করা হয়, এটি প্রক্রিয়ায় একটি thread context তৈরি করবে এবং এই thread context বা এই execution context এই বিশেষ ফাংশন থেকে চালানো শুরু হবে। এখন এই নির্দিষ্ট thread এ argument টি প্রবর্তন করা হবে যেমন pthread create সর্বশেষ argument হিসাবে নির্দিষ্ট করা হবে। এইভাবে আমরা 4 টি pthread underscore create করেছি এবং এইভাবে একক প্রক্রিয়ার মধ্যে 4 টি thread তৈরি করেছি। এই সমস্ত 4 টি thread একই প্রারম্ভিক ফাংশন আছে যা thread underscore fn এবং thread ID গুলি এই প্রথম parameter এ fill করা হবে যা t 1, t 2, t 3, এবং t 4 । এখন এই 4 টি thread গুলির প্রত্যেকটি এই থ্রেড ফাংশন থেকে এক্সিকিউট করা শুরু করবে, যা আপনি মনে করতে পারেন main thread এর মতো, arg এর ভিন্ন মান দেখতে পাবেন। উদাহরণস্বরূপ, thread 1 এর 0 একটি arg value থাকবে, thread 2 এর 1 এর একটি arg value থাকবে, thread 3 এর একটি arg value 2 হবে এবং thread 4 এর 3 এর একটি arg value থাকবে। সুতরাং, এই arg মানটি ব্যবহার করে তারপর ID তে type cast করা হয় এবং তারপর এটি আসলে start নির্ধারণ করতে ব্যবহৃত হয়। সুতরাং, শুরু করার পরে নির্দিষ্ট thread 2.5 মিলিয়ন নম্বর কোনটি যোগ করা উচিত তা নির্ধারণ করতে ব্যবহার করা হয়। উদাহরণস্বরূপ যদি একটি thread arg 0 হয়, তাহলে type casting এর পরে ID এর মানটি 0 এবং stack মান 0 হবে। তাই, ID 0 এর সাথে এই thread টি প্রথম 2.5 মিলিয়ন সংখ্যা যোগ করবে যা 0 থেকে সংখ্যা 2.5 মিলিয়ন। এখন, এই ছাড়াও, সংজ্ঞায়িত করা হয় একটি sum নামে global array। সুতরাং, এটি 4 টি element এর একটি unsigned long array । এখন এই যোগফলটি প্রতিটি thread এর সমষ্টি যোগ করার জন্য ব্যবহৃত হয়। সুতরাং, sum 0 0 থেকে 2.5 মিলিয়ন হতে হবে যা thread 0 অথবা T1 দ্বারা পূর্ণ হয় । একইভাবে, sum 1 সংখ্যা 2.5 মিলিয়ন থেকে 5 মিলিয়ন এবং এর একটি যোগফল thread T2 দ্বারা পরিপূর্ণ হবে। অনুরূপভাবে sum 2 এবং sum 3 এর সংশ্লিষ্ট 2.5 মিলিয়ন যোগফল হবে এবং এটি T3 এবং T4 দ্বারা পরিপূর্ণ তাই, এই প্রক্রিয়াটি 4 টি thread এর মধ্যে চলছে যখন এই লুপটি দীর্ঘ সময় ধরে নেয়, যখন 4 টি thread স্বাধীনভাবে চালানো হচ্ছে। 4 টি thread তৈরির পর main thread join কে invoke করবে। তাই, এটি 4 টি thread এর সাথে 4 টি যোগফল যোগ করে এবং এই যোগদানটি কি করবে তা হল যে এটি TID দ্বারা নির্ধারিত সুনির্দিষ্ট thread টি শেষ না হওয়া পর্যন্ত এটি blocked থাকবে। এইভাবে যখন thread T1 এই বিশেষ ফাংশন থেকে প্রস্থান করে, এই pthread underscore join wake up করবে এবং এটি এক্সিকিউশন সম্পূর্ণ করবে। তাই, এইভাবে 4 টি কাজ শেষ করার সাথে সাথে ফাংশনগুলি যোগ হবে যে সমস্ত 4 টি thread তাদের execution কার্যকর করেছে। এরপর 4 টি thread থ্রেড exit হয়ে

যাওয়ার পরে, 2.5 মিলিয়ন সংখ্যার আংশিক অঙ্কের sum যোগ করা হয় এবং চূড়ান্ত ফলাফল পেতে printf ব্যবহার করে print হয়। সুতরাং, এই final sum হল printed প্রথম 10 মিলিয়ন সংখ্যার সমষ্টি এবং তাই, আমরা দেখেছি যে 0 থেকে 10 মিলিয়ন থেকে সংখ্যা যোগ করার বড় কাজটি 4 টি ভাগে বিভক্ত এবং প্রতিটি অংশ তারপর কাজ করে তারপর একটি দ্বারা গণনা করা হয় থ্রেড একটি আংশিক ফলাফল পেতে তারপর যা যোগ করা হয়। এখন এই প্রোগ্রাম চালানোর জন্য আমরা gcc thread dot c ব্যবহার করি যা compilation এবং একটি minus lpthread নির্দিষ্ট করে যা pthread লাইব্রেরি, যা এই বিশেষ executable এর সাথে সংযুক্ত করা প্রয়োজন। সুতরাং, gcc এর ফলে একটি a dot out executable তৈরি হয় যা তারপর execute করা হয়। সুতরাং, আপনি আসলে একটি লিনাক্স সিস্টেমের উপর এই বিশেষ প্রোগ্রামটি চেষ্টা করে দেখতে পারেন এবং আপনি বিভিন্ন প্রসেসের এবং একটি sequential প্রোগ্রাম যেখানে শুধুমাত্র এই কাজটি সম্পাদন করে তার তুলনায় বিভিন্ন thread এর মাধ্যমে আপনি কতটা দ্রুত পেতে পারেন তা নির্ধারণ করার চেষ্টা করতে পারেন।

(স্লাইড সময় : 36:04 দেখুন)

Pthreads ছাড়াও বেশ কয়েকটি লাইব্রেরি রয়েছে যা আপনি multi threaded program তৈরি করতে ব্যবহার করতে পারেন। উদাহরণস্বরূপ, Windows Thread Library, Microsoft Windows ভিত্তিক অ্যাপ্লিকেশনের জন্য ব্যবহৃত একটি লাইব্রেরি যা Boost C ++ এর জন্য একটি লাইব্রেরি এবং Linux thread নামেও আরেকটি লাইব্রেরি রয়েছে। সুতরাং, যদি আমরা আসলে thread লাইব্রেরিগুলির জন্য Google করি তবে এমন একটি লাইব্রেরির একটি বড় তালিকা পাবেন যা উপস্থিত রয়েছে। ধন্যবাদ।